

Setting Up Claude to Edit a Client Dashboard

A first-time setup guide — no coding experience needed. By the end, you'll be able to ask Claude to make a change to a client's dashboard, review it, and push it live.

This whole process has three ongoing steps once you're set up: **you ask Claude for a change**, **Claude edits the files on your computer**, and **you push the change to GitHub** (which automatically updates the live site). Everything before that is one-time setup — you only do it once.

1 Install Git

Git is the free program that lets your computer talk to GitHub (where the dashboard's code is stored). You need this installed before anything else works.

1. Go to `https://git-scm.com/download/win` in your browser. The download starts automatically.
2. Open the downloaded file and click **Next** on every screen, keeping all the default options. You don't need to change any settings.
3. Click **Install**, then **Finish**.

That's it — you won't open a "Git" program directly. It works quietly in the background whenever you type a `git` command in Command Prompt.

2 Create a folder for the project

This is the folder on your computer where the dashboard's code will live, and where Claude will make edits.

1. On your Desktop (or wherever you like), create a new folder.
2. Name it something clear, like `{Client Name} Vercel Project` — swap in the actual client's name.
3. Keep this folder — don't rename or move it later, since Claude and your terminal will both point at it.

3 Copy the project code into that folder

Next you'll pull a copy of the dashboard's code from GitHub into your new folder. This is called "cloning."

1. Open **Command Prompt** (press the Windows key, type `cmd`, press Enter).
2. Type the following, replacing the path with your own folder's location, and press Enter:

```
cd "C:\Users\YourName\Desktop\{Client Name} Vercel Project"
```

3. Then type this and press Enter (ask your manager for this client's actual repo URL — it'll look like this, just with a different client name in it):

```
git clone https://github.com/your-org/{client}-dashboard.git .
```

The dot (`.`) at the end matters — it tells Git to put the files directly into the folder you're already in, instead of making a new sub-folder.

When it's done, open the folder in File Explorer — you should see files like `package.json`, `app`, and `components` inside it.

4 Connect the folder to Claude

Open Cowork (Claude's desktop app) and tell it to work on this folder.

1. Start a new chat in Cowork.
2. Ask Claude to connect to your folder — for example: *"Please connect to my folder at C:\Users\YourName\Desktop\{Client Name} Vercel Project".*
3. Claude will ask you to approve access — click to allow it.

Once it's connected, Claude can read and edit every file in that folder directly — you don't need to open or copy files yourself.

5 Ask Claude to make a change

Just describe what you want in plain English. For example:

```
// example request, type this to Claude:  
"Can you add a new tab called Payroll to the dashboard? It should show..."
```

Claude will read the existing code, make the change, and tell you what it did. You don't need to understand the code — just read Claude's summary of the change.

6 Look at the change (optional but recommended)

Before pushing, it's good practice to open the dashboard locally to see the change with your own eyes. Ask Claude, or run this yourself from Command Prompt in that same folder:

```
npm run dev
```

Then open `http://localhost:3000` in your browser to preview it. Press `Ctrl+C` in Command Prompt when you're done to stop it.

7 Push it — send the change live

"Pushing" uploads your change to GitHub, which automatically updates the live Vercel site within a minute or two. This is the step to remember.

Important: always type these commands yourself in Command Prompt. Never ask Claude to run `git` commands for you, even if it offers to. Claude works in its own separate sandbox — if it runs `git` against your folder directly, it can leave a stuck `index.lock` file behind (see Troubleshooting below) that blocks your next push. Claude editing files is fine; Claude running `git` is not.

In Command Prompt, make sure you're in the project folder, then run these three commands, one at a time:

```
cd "C:\Users\YourName\Desktop\{Client Name} Vercel Project"
git add -A
git commit -m "Describe what changed here"
git push
```

1. `git add -A` — gathers up every file Claude changed.
2. `git commit -m "..."` — saves those changes with a short note about what changed (write your own short description in the quotes).
3. `git push` — uploads everything to GitHub.

Tip: before you commit, run `git status` on its own first. It lists every file that changed — a quick sanity check that Claude's edit is actually there before you push it.

8 Check it went live

Open the dashboard's live web address in your browser (ask your manager for the exact link) and do a hard refresh: `Ctrl + Shift + R`. Give it a minute or two after pushing — Vercel needs a short time to rebuild the site.

Quick reference — the 3 commands you'll use every time

1. See what changed

```
git status
```

2. Stage the changes

```
git add -A
```

3. Save with a note

```
git commit -m "your note"
```

4. Send it live

```
git push
```

Troubleshooting

"fatal: Unable to create './.git/index.lock': File exists" / "Another git process seems to be running"

This is the most common snag. It means a leftover file called `index.lock` got stuck inside the hidden `.git` folder, and Git refuses to run until it's gone. It almost always happens because a git command got interrupted or run from somewhere it shouldn't have been — which is exactly why Step 7 says to always run git yourself, never through Claude.

1. In File Explorer, open your project folder.
2. Turn on hidden files: click **View** at the top, then check **Hidden items**.
3. Open the `.git` folder (it will look slightly greyed out).
4. Find the file named `index.lock` and delete just that one file.

Name	Status	Date modified	Type	Size
hooks	📁	7/30/2026 5:35 PM	File folder	
info	📁	7/30/2026 5:35 PM	File folder	
logs	🔄	7/30/2026 5:37 PM	File folder	
objects	📁	7/30/2026 5:35 PM	File folder	
refs	📁	7/30/2026 5:35 PM	File folder	
config	🔄	7/30/2026 5:37 PM	File	1 KB
description	✅	7/30/2026 5:35 PM	File	1 KB
HEAD	🔄	7/30/2026 5:35 PM	File	1 KB
index	🔄	7/30/2026 5:37 PM	File	4 KB
index.lock	🔄	7/30/2026 5:37 PM	LOCK File	0 KB
ORIG_HEAD	🔄	7/30/2026 5:37 PM	File	1 KB
packed-refs	🔄	7/30/2026 5:35 PM	File	1 KB

The `.git` folder with `index.lock` selected — delete this file, nothing else in this folder.

Then try your `git commit` / `git push` command again.

Important: only ever delete `index.lock` itself — never delete the whole `.git` folder. That folder is the project's entire history; if it ever seems broken beyond this one file, stop and ask your manager or Claude before deleting anything else in there.

"Everything up-to-date" after you push, but you don't see your change

This means `git commit` didn't actually save anything (usually because of the lock-file problem above happening one step earlier and going unnoticed). Run `git status` — if it lists changed files, redo Step 7. If it says "nothing to commit," ask Claude to double check the files were actually saved.

You pushed, but the live site still looks old after a few minutes

Make sure you're checking the right web address — a specific deployment link (one with a long random code in it, like `...-1iu33cc1j-...vercel.app`) never updates. Use the main project link instead (ask your manager which one that is), and do a hard refresh (`Ctrl+Shift+R`).